AI-BASED EXAM TIMETABLE GENERATOR

FOR

Foundations of Artificial Intelligence

AI-BASED EXAM TIMETABLE GENERATOR USING CONSTRAINT SATISFACTION, BACKTRACKING SEARCH AND HEURISTICS

A PROJECT REPORT

Submitted by

2116240701049 – ARJUNRAJ M

2116240701057 – ASHWANTH K



October, 2026

BONAFIDE CERTIFICATE

Certified that this project report "AI-BASED EXAM TIMETABLE GENERATOR" is the bonafide work of "ARJUNRAJ M (240701049) and ASHWANTH K (240701057)" who carried out the project work under my supervision.

SIGNATURE OF THE FACULTY INCHARGE

Submitted for the Practical Examination held on _____

SIGNATURE OF THE INTERNAL EXAMINER

TABLE OF CONTENTS

CHAPTER NO.	TITLE	PAGE NO.
	ABSTRACT	4
1	INTRODUCTION	5
2	LITERATURE REVIEW	7
3	PROPOSED SYSTEM	8
	3.1 ARCHITECTURE DIAGRAM	8
4	MODULES DESCRIPTION	10
	4.1 MODULE 1: DATA INPUT & CSP MODELLING	10
	4.2 MODULE 2: PROPAGATION & BACKTRACKING	10
	4.3 MODULE 3: OPTIMIZATION & SCORING	10
	4.4 MODULE 4: CONFLICT CHECKER	10
5	IMPLEMENTATION AND RESULTS	11
	5.1 EXPERIMENTAL SETUP	11
	5.2 RESULTS	11
6	CONCLUSION AND FUTURE WORK	12
7	REFERENCES	13

ABSTRACT

Manual examination timetabling is a slow, error-prone process that struggles to satisfy the many overlapping constraints imposed by students, faculty, rooms and time slots. This project models exam scheduling as a Constraint Satisfaction Problem (CSP), where each exam is a variable and each available slot is a candidate value in its domain.

A backtracking search, guided by the Minimum Remaining Values (MRV) heuristic and strengthened with constraint propagation, assigns exams to slots so that hard constraints — student, faculty, room and capacity conflicts — are never violated, while soft constraints such as spacing out difficult exams and balancing the daily load are minimized.

The result is an admin-facing web application that ingests subjects, rooms and the exam period, then automatically generates an optimized, conflict-free timetable — complete with a quantified optimization score and a plain-language explanation of why one candidate timetable beats another. The sample run achieved an optimization score of 94/100 with full hard-constraint compliance.

CHAPTER 1

INTRODUCTION

1.1 Background

Every semester, colleges must schedule end-semester examinations for many subjects and sections across a limited number of days, rooms and invigilators. A valid timetable must ensure that no student has two exams at the same time, no faculty member is assigned to two places at once, no room is double-booked and no room is filled beyond its capacity.

Examination timetabling is a well-known NP-complete problem. As the number of subjects, sections and rooms grows, the number of possible schedules grows exponentially, which makes purely manual scheduling impractical and error-prone.

1.2 Problem Statement

Most colleges still build exam timetables manually or with basic spreadsheets. There is no automated reasoning over constraints, so conflicts are found late, difficult exams may fall on consecutive days, and any late change forces large parts of the schedule to be redone. This project aims to build an AI-based system that automatically generates conflict-free and well-balanced exam timetables.

1.3 Objectives

- Model exam timetabling as a Constraint Satisfaction Problem with exams as variables and slots as domain values.
- Guarantee that hard constraints (student, faculty, room and capacity conflicts) are never violated.
- Use backtracking search with the MRV heuristic and constraint propagation to reduce search effort.

- Minimize soft-constraint violations such as back-to-back difficult exams and uneven daily load.
- Provide a quantified optimization score and an explanation when comparing candidate timetables.
- Allow quick re-planning when subjects, rooms or dates change.

1.4 Scope

The system is intended for use by examination coordinators and administrators. It accepts subject, room and exam-period data as input and produces a final timetable together with a conflict report. It is designed as an admin-facing web application.

1.5 Existing System

In the existing system, timetables are prepared by hand or in spreadsheets. Its main limitations are:

- **Manual, spreadsheet-driven:** Coordinators cross-check students, faculty and rooms manually, with no automated conflict detection.
- **No constraint guarantees:** Nothing prevents a student, faculty member or room from being double-booked until someone notices.
- **Ignores room capacity and type:** Capacity limits and special-room needs (labs, halls) are not checked systematically.
- **No difficulty-load optimization:** Difficult exams may fall on consecutive days purely by chance.
- **Slow to re-plan:** A single late change forces large parts of the schedule to be redone by hand.
- **Not scalable:** Manual scheduling time and errors grow rapidly with more subjects, sections and rooms.

CHAPTER 2

LITERATURE REVIEW

Examination timetabling has been studied extensively in operations research and artificial intelligence. The works reviewed for this project are summarized below.

Author(s) / Year	Focus and Method	Key Takeaway
McCollum et al. (ITC 2007)	Proposed a realistic hard/soft constraint benchmark model for examination timetabling, used as a competition dataset.	Established the standard hard-vs-soft constraint formulation on which this project's constraint set is based.
Boufflet and Nègre – CLP for Examination Timetabling	Applied Constraint Logic Programming to schedule about 4,000 students' exams across limited rooms and half-day slots.	Confirmed that exam scheduling is NP-complete and well suited to constraint-based solving.
Sharma and Sharma – ICHEA (2020)	A constraint-guided evolutionary algorithm that satisfies hard constraints first, then optimizes soft objectives.	Motivated the two-stage approach: satisfy hard constraints via CSP, then optimize soft ones.
CSP Study, University of Sharjah	Modelled a real college's exam timetable as a CSP solved with Constraint Logic Programming (CLP/OPL).	Showed that CSP formulations scale to real, resource-limited institutional timetables.
Tsang – Constraint Satisfaction: Algorithms and Applications	Foundational survey framing timetabling as a classic constraint satisfaction problem solvable via search.	Provides the theoretical basis for the variables, domains and constraints model used here.

From the survey, it is evident that constraint-based approaches handle hard constraints reliably, while local search or evolutionary methods are useful for optimizing soft preferences. This project combines both ideas: a CSP backtracking solver guarantees feasibility, and a soft-constraint optimization step improves quality.

CHAPTER 3

PROPOSED SYSTEM

The proposed system is an AI scheduling engine that treats timetabling as a CSP and reasons over every constraint before any slot is fixed.

- **CSP-based modelling:** Every exam is a variable and every slot is a domain value; constraints are declared once rather than checked by hand.
- **Backtracking with MRV heuristic:** The engine always resolves the most-constrained exam first, cutting wasted search and finding valid schedules faster.
- **Hard constraints never violated:** Student, faculty, room and capacity conflicts are eliminated by construction, not caught after the fact.
- **Soft-constraint optimization:** Local search minimizes back-to-back difficult exams and balances load, scoring each candidate timetable.
- **Instant re-planning:** Any change to subjects, rooms or dates can be re-solved in seconds.
- **Explainable comparisons:** The system can generate alternative timetables and explain in plain language why one scores higher.

3.1 Architecture Diagram

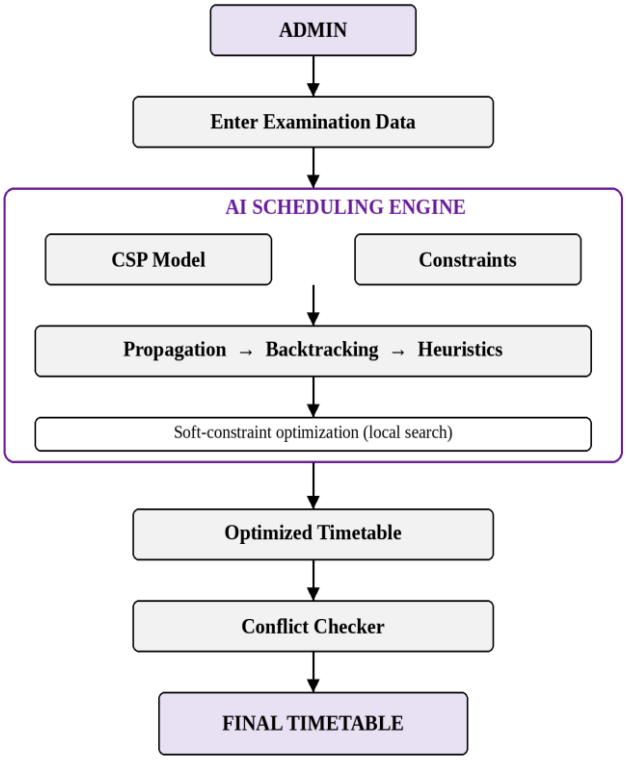


Figure 3.1 System Architecture

The administrator enters examination data (subjects, rooms, exam period). The AI scheduling engine builds the CSP model and its constraints, then applies propagation, backtracking and heuristics to produce a candidate timetable. The optimized timetable is verified by a conflict checker before the final timetable is released.

3.2 Constraints

Type	Constraint
Hard	No student may have two exams in the same slot (student conflict).
Hard	No faculty member may be assigned to two exams in the same slot (faculty conflict).
Hard	No room may host two exams in the same slot (room conflict).
Hard	The number of students must not exceed room capacity, and special-room needs (labs, halls) must be met.
Soft	Avoid back-to-back difficult exams for the same student group.

Type	Constraint
Soft	Balance the number of exams scheduled per day.

CHAPTER 4

MODULES DESCRIPTION

4.1 Module 1: Data Input and CSP Modelling

The administrator enters subjects, student groups, faculty, rooms with capacities and types, and the exam period. This module converts the input into a CSP: each exam is a variable, the domain of each variable is the set of available (day, slot, room) combinations, and the hard constraints are registered as pairwise constraints between exams.

4.2 Module 2: Constraint Propagation and Backtracking Search

This is the core solver. Before and during search, constraint propagation removes values from domains that can no longer be valid. The Minimum Remaining Values (MRV) heuristic then selects the exam with the fewest remaining legal slots, since it is the most likely to cause failure. A value is assigned, propagation is repeated, and if a dead end is reached the search backtracks and tries another value. This ensures that hard constraints are never violated in the final result.

4.3 Module 3: Soft-Constraint Optimization and Scoring

Once a feasible timetable exists, a local search step improves its quality by moving or swapping exams so that difficult exams are spaced out and the daily load is balanced, without breaking any hard constraint. Each timetable receives an optimization score out of 100 based on the soft-constraint penalties.

4.4 Module 4: Conflict Checker and Explanation

The conflict checker independently verifies the final timetable against every hard constraint and reports any violation. The explanation component compares candidate timetables by their scores and states in plain language why one is

better, for example fewer back-to-back difficult exams or a more even daily load.

CHAPTER 5

IMPLEMENTATION AND RESULTS

5.1 Experimental Setup

The system is implemented as an admin-facing web application in which the administrator provides the following inputs:

- List of subjects with their student groups, assigned faculty and difficulty level.
- List of rooms with capacity and type (classroom, lab, hall).
- Examination period (start date, end date and available slots per day).

The scheduling engine runs the CSP solver with MRV-guided backtracking and propagation, followed by soft-constraint optimization. Output is validated by the conflict checker.

5.2 Results

The generated timetable satisfied all hard constraints; no student, faculty or room conflicts and no capacity violations were present. The sample run achieved the following:

Parameter	**Outcome**
Solving approach	CSP – Backtracking + MRV heuristic
Hard-constraint compliance	100% (target met)
Sample optimization score	94 / 100
Re-planning after a change	Re-solved automatically in seconds

Compared with the manual approach, the proposed system removes late discovery of conflicts, spaces out difficult exams, and makes re-planning fast. Alternative timetables can be generated and compared through their scores, with a plain-language explanation of the difference.

CHAPTER 6

CONCLUSION AND FUTURE WORK

6.1 Conclusion

This project presented an AI-based exam timetable generator that models scheduling as a Constraint Satisfaction Problem and solves it with backtracking search, the MRV heuristic and constraint propagation. Hard constraints are satisfied by construction, and soft constraints are optimized through local search with a quantified score. The system replaces slow, error-prone manual scheduling with a fast, explainable and easily re-plannable process.

6.2 Future Work

- Add further heuristics such as degree and least-constraining-value ordering, and stronger consistency checks such as arc consistency.
- Support invigilator allocation and seat-wise arrangement within rooms.
- Import student enrolment data directly from the college database.
- Support multi-campus scheduling and larger datasets with performance benchmarking.
- Provide notifications to students and faculty once the timetable is published.

CHAPTER 7

REFERENCES

Boufflet, J.-P. and Nègre, S. – Constraint Logic Programming for Examination Timetabling.

McCollum, B. et al. (2007) International Timetabling Competition 2007 (ITC 2007): Examination Timetabling Track.

Sharma, A. and Sharma, S. (2020) Constraint-guided Evolutionary Algorithm (ICHEA) for Examination Timetabling.

Tsang, E. (1993) Foundations of Constraint Satisfaction, Academic Press.

University of Sharjah – Examination Timetabling as a Constraint Satisfaction Problem using Constraint Logic Programming (OPL).